

Git, the short version

Git is how you get the course material and keep it up to date. This page is all you need for that.

Once, when you install it

`git --version` Checks Git is installed. If this errors, install Git and reopen your terminal.

`git config --global user.name "Your Name"` Tells Git who you are. It stamps this on anything you save.

`git config --global user.email "you@univ.fr"` Same, with your university address. Once per machine, not per project.

Once, to get the course

`git clone <course-url>` Downloads the whole course into a new folder on your machine.

`cd <folder-name>` Moves into that folder. Every command below is run from inside it.

course-url: <https://github.com/<your-account>/<course-repo>.git>

Every week, before the session

`git pull` Brings in the sessions published since you last looked. This is the one command you will actually use every week.

THE ONE RULE

Never work directly in the files that came from the course. Duplicate the notebook first.

In JupyterLab: right-click the notebook, choose Duplicate, then rename your copy, for example 01_workflow_MY_WORK.ipynb.

Why it matters: git pull refuses to run when you have changed a file that the course has also changed. If all your work lives in copies you made, that never happens, you never lose work, and you avoid almost every Git problem a student on this course will meet.

When git pull refuses

`error: Your local changes to the following files would be overwritten by merge`

You edited a course file. Nothing is broken and nothing is lost. Pick either way out:

`git stash`
`git pull`
`git stash pop` Keep your edits. Puts them aside, updates the course, then puts your edits back on top.

`git restore .`
`git pull` Discard your edits to course files, then update. Your own duplicated notebooks are untouched, because Git is not tracking them. On older Git, use `git checkout -- .` instead.

When something surprises you

You will not need this page most weeks. Keep it for the week you do.

Seeing where you are

`git status`

What has changed, and whether you are behind the course repository. Run this first, always.

`git log --oneline -5`

The last five updates, one line each, newest first.

`git diff`

Exactly which lines changed, before you decide what to do about them.

Using Git for your own project work

Optional, and it needs a repository of your own: the course repository is read-only for you.

`git add <file>`

Mark a file to be included in the next snapshot.

`git commit -m "what I did"`

Save a snapshot of the marked files, with a short message.

`git push`

Send your saved snapshots to your own repository online.

If something breaks

`git restore <file>`

Undo your uncommitted changes to one file, and only that file.

`git commit --amend -m "better"`

Fix the message on your most recent commit. Only if you have not pushed it yet.

IF YOU ARE TRULY STUCK

Clone the course again into a new folder, and copy your own notebooks across.

Two minutes, and it always works: you lose nothing, because your work lives in your copies and not in the course files. One command to avoid entirely is `git push --force` on anything shared, which rewrites history for everyone else too.

Words you will hear

repository

A folder Git tracks, with its full history.

commit

One saved snapshot, with a message describing it.

pull

Bring changes from the remote down to your machine.

clone

Make your own complete copy of a repository.

remote / origin

The copy online. origin is its usual nickname.

push

Send your commits from your machine up to the remote.

Page 1 is the whole job: clone once, pull weekly, and always work in your own copy of a notebook.